Research 15 min read

When the Memory Wall Disappears: What Actually Bottlenecks LLM Inference on Modern GPUs

ai.rs Mar 5, 2026

cuda inference gpu quantization kernel-fusion benchmarks

ASIC chips designed for LLM inference are arriving. Groq's LPU, Cerebras's WSE, and a wave of startups are all chasing the same insight: autoregressive

token generation is memory-bound, so bu with massive on-chip SRAM and skip the DRAM bottleneck entir h is compelling — if your weights live on-chip, you eliminate the memory wall and inference becomes compute-limited.

But here's a question worth asking: what happens when you simulate this on a commodity GPU today? NVIDIA's RTX 5090 ships with 96 MB of L2 cache. A quantized 135M-parameter model fits in 85 MB. If you pin those weights in L2, you've effectively built a poor man's ASIC — all weights on-chip, no DRAM round-trips during generation.

This article documents what we found when we tried it. Spoiler: the memory wall does disappear. What replaces it is more interesting.

The Setup: SmolLM2-135M on RTX 5090

We built a custom CUDA inference engine from scratch for SmolLM2-135M, a 30-layer transformer with 576-dimensional hidden state, 9 query heads, 3 KV heads (GQA), and a 1536-dimensional FFN. The architecture is standard — RMSNorm, RoPE, grouped-query attention, SwiGLU MLP — just small enough to be interesting.

The model's weights are stored in GGUF's IQ4_NL and IQ4_XS quantization formats. IQ4_NL packs 32 values into 18 bytes: a half-precision scale factor and 16 bytes of 4-bit indices into a non-linear lookup table. The lookup table lives in CUDA constant memory for broadcast access:

```
__device__ __constant__ float d_kvalues_iq4nl[16] = {
    -127.f, -104.f, -83.f, -65.f, -49.f, -35.f, -22.f, -10.f,
       1.f,   13.f,  25.f,  38.f,  53.f,  69.f,  89.f, 113.f
};
```

The total weight pool — all 30 layers of IQ4_NL/IQ4_XS projections, Q8_0 embeddings, FP16 norms — comes to 85 MB.

The RTX 5090 (Blackwell, SM 12.0) has 96 MB of L2 cache. At engine startup, we pin the weight pool into L2 using `cudaStreamSetAttribute` with `cudaAccessPolicyWindow`:

Property	Value
GPU	RTX 5090 (Blackwell)
VRAM	32 GB GDDR7, 1,790 GB/s
L2 cache	96 MB
Weight pool	85 MB (IQ4_NL/IQ4_XS + Q8_0)
L2 hit ratio	~100% during generation

Once the weights are warm in L2, every mat-vec reads from on-chip cache. No DRAM traffic for weights. This is the ASIC scenario.

Phase 1: Naive FP16 — 750 tok/s

The first version used FP16 weights and straightforward kernels: one RMSNorm, one mat-vec per projection, separate RoPE, separate KV cache writes. This was the baseline to validate correctness.

At 750 tok/s for 128-token generation, it was already faster than running the same model under most Python-based frameworks, but well below llama.cpp's 1,110 tok/s. The FP16 weight pool was too large for L2 pinning, so this phase still hit DRAM for weights.

Phase 2: IQ4 Quantization + L2 Pinning — 1,255 tok/s

Switching to IQ4_NL/IQ4_XS quantization (loaded directly from GGUF, no conversion) shrunk the weight pool from ~270 MB to 85 MB. Now it fits in L2.

The mat-vec kernel design uses one warp (32 threads) per output row. Each warp iterates over IQ4 blocks, dequantizing through the shared-memory lookup table and accumulating a dot product against the input vector (also in shared memory). The warp reduction is a standard shuffle tree:

```
template<bool FUSE_RESIDUAL = false>
__global__ void matvec_iq4nl(half* __restrict__ out,
                             const void* __restrict__ W,
                             const half* __restrict__ x,
                             half* __restrict__ residual,
                             int out_dim, int in_dim) {
    // ... cooperative load of x into shared memory ...
    const int row = blockIdx.x * warps_per_block + warp_id;
    float sum = 0.0f;
    for (int b = 0; b < blocks_per_row; b++) {
        float d = __half2float(row_blocks[b].d);
        uint8_t q = row_blocks[b].qs[lane & 15];
        int shift = (lane >> 4) << 2;
        int idx = (q >> shift) & 0xf;
        float w = d * s_kv[idx];
        sum += w * s_x[b * 32 + lane];
    }
    // warp shuffle reduction ...
}
```

With L2 pinning, this hit 1,255 tok/s. A 67% improvement over FP16, mostly from the L2 effect — weights served at L2 bandwidth (~3-4 TB/s effective) instead of DRAM (1,790 GB/s peak).

At this point, the memory wall was gone. Now what?

The natural instinct was to optimize the compute. IQ4_NL dequantization requires a shared-memory table lookup — what if we converted everything to Q8_0 at load time? Q8_0 dequant is a simple `d * qs[i]`, no lookup needed.

We tried it. Mat-vec bandwidth improved from 95 to 152 GB/s. But tok/s barely moved: 1,255 to 1,262.

Why? Two reasons. First, Q8_0 is 34 bytes per 32 values vs. IQ4_NL's 18 bytes. The weight pool grew from 85 to 136 MB — too large for L2 pinning. We traded lookup latency for cache misses. Second, and more fundamental: the layer matrices are tiny. The largest FFN projection is 1536 rows of 576 elements. At that size, a single mat-vec completes in microseconds regardless of dequant cost. The kernel finishes before the GPU has time to be bottlenecked on anything.

The real bottleneck was hiding in the profile output. Each forward pass launched 301 kernels. Each kernel launch costs ~2.5 microseconds of driver overhead. That's 750 microseconds of pure launch tax — almost the entire per-token time budget of 792 microseconds.

The memory wall was gone. The dispatch wall had replaced it.

Phase 3: Kernel Fusion — 1,508 tok/s

Once we identified dispatch overhead as the bottleneck, the optimization strategy flipped. Instead of making individual kernels faster, we needed fewer of them. Each of the 30 layers ran 10 kernels. We fused them down to 6.

Fusion 1: Residual Addition into Mat-Vec

After the attention output projection and the [illegible] projection, the original code ran a separate `vec_add` kernel to acc[illegible] the residual:

```
// Before: two kernel launches
matvec_iq4nl<<< ... >>>(xb, attn_output, attn_out, nullptr, DIM, DIM);
vec_add<<< ... >>>(x, x, xb, DIM);
```

The vec_add kernel reads and writes 576 half values. It takes about 2 microseconds of compute but 2.5 microseconds to launch. We added a template parameter to the mat-vec kernel:

```
if (lane == 0) {
    float result = sum;
    if constexpr (FUSE_RESIDUAL) {
        result += __half2float(residual[row]);
        residual[row] = __float2half(result);
    }
    out[row] = __float2half(result);
}
```

[illegible]

[illegible]

Continue reading

One OAuth click to unlock [illegible] plus downloadable resources and [illegible] updates. No spam, no paywall.

Subscribe with email to continue reading

Wondering if this fits your business?

The benchmarks and architectures we cover here power real production AI assistants. See where your business stands in 2 minutes.

Take the AI Readiness Check →

Share: Post Share Copy link

Previous
The GPU Memory Wall: Why Inference Hardware Matters

Next
From Edge AI to Custom LLMs: How On-Device Intelligence Evolved

Read next

Research

The GPU Memory Wall: Why Inference Hardware Matters

GPU cores sit idle 98% of the time during LLM inference. Understanding...

AI For Business

100% ROI in 24 Hours: Nvidia B200 Replaced a $35,000 AI API...

We needed AI-generated SEO descriptions for 858,000 products. Th...

15 min read

ai.rs AI for Business AI Beginner Developer Corner How It Works Ctrl+K

AI News

Qwen 3.5: 35B Knowledge at 4B Speed — Better Than GPT-5?

Alibaba's Qwen 3.5 drops 8 models in two weeks — from 0.8B to 397B...

12 min read

Stay updated with AI insights — no spam, unsubscribe anytime.

Telegram your@email.com Notify

ai.rs — AI That Works for Your Business

© 2026 ai.rs. Built with research, not hype.

Get Started